CYBER ATTACK PREDICTION PLATFORM USING SPATIO-TEMPORAL GRAPH NEURAL NETWORKS



Dept. of Computer Science & Engineering
College of Engineering, Karunagappally (CEK)

Under the Guidance of
Mrs. SYAMA S R
Assistant Professor, Dept. of CSE

Project Group 14

MUHAMMED NASIH K P (KNP23CS074)
SUMAYYA ERSHAD (KNP23CS107)
NAJMUNNIZA N (LKNP20CS056)
K.S. MAHADEV (KNP23CS066)

Presentation Overview

Part I: Foundations, Review & Core Methodology

- **Phase 1: Foundations & Problem Formulation**
  - 1. Introduction & Motivation
  - 2. Problem Statement (SOC Challenges)
  - 3. Objectives & Target Milestones
  - 4. Literature Review (16 Base Papers)
  - 5. Research Gaps & Proposed Solutions
- **Phase 2: System Architecture & Workflow**
  - 6. System Design Principles & Benchmarks
  - 7. End-to-End Workflow Diagram
- **Phase 3: Multi-Model Detection Pipeline**
  - 8. Proposed Methods & Formulations
  - 9. Multi-Model Architecture & Roles
  - 10. End-to-End Inference Flow

Part II: Data, Schemas, Algorithms & Management

- **Phase 4: Data Engineering & Model Training**
  - 11. Dataset Preparation & Loop Compaction
  - 12. Model Training, Calibration & Evaluation
- **Phase 5: Deep Schemas & Core Algorithms**
  - 13. System Architecture Diagram
  - 14. Development & Testing Flowchart
  - 15. Dual-Tier Database Schemas
  - 16. Core Algorithms & Formal Pseudocode
- **Phase 6: Implementation, Project Plan & Conclusion**
  - 17. Implementation & Technology Stack
  - 18. Project Work Plan & Gantt Roadmap
  - 19. Conclusion & Future Scope
  - 20. References (Academic Literature)

1. Introduction

Modern enterprise networks face hidden cyberattacks called **Advanced Persistent Threats (APTs)**, which avoid traditional security systems by behaving like normal users over long periods. Traditional intrusion detection systems often analyze logs separately, missing important connections between processes, files, and network connections (**Process** $\rightarrow$ **File** $\rightarrow$ **Socket**).

Our system represents system activities as **dynamic provenance graphs** and combines **Graph Attention Networks (GAT)**, **Gated Recurrent Units (GRU)**, and **Variational Graph Autoencoders (VGAE)** to learn normal behavior and detect **unknown attacks without labeled training data**. It also uses **TenSEAL CKKS-encrypted Federated Learning** to enable secure collaboration between organizations while protecting their data.

2. Problem Statement

Traditional security systems struggle to detect **stealthy, multi-stage cyberattacks** because they **analyze system logs separately**, miss complex relationships and time-based attack patterns, and generate **false alarms** from normal activities. They also struggle to detect **unknown zero-day threats** while protecting sensitive data.

This project proposes a **spatio-temporal graph-based anomaly detection framework** using **GAT**, **GRU**, and **VGAE** to detect suspicious activities, **reconstruct attack paths**, map threats to the **MITRE ATT&CK framework**, and enable **privacy-preserving collaborative detection**.

3. Objectives

The main objective of this project is to develop an **intelligent and privacy-preserving cybersecurity system** that detects **hidden, multi-stage APT attacks** and **traces their attack paths** across enterprise networks.

The system converts raw kernel audit logs from the **DARPA Transparent Computing dataset** into **provenance graphs** and uses **GAT, GRU, and VGAE** to **detect unknown attacks** by identifying unusual graph patterns. It also supports secure collaborative training using **TenSEAL CKKS encryption**, reconstructs attack paths across multiple computers, maps detected threats to the **MITRE ATT&CK framework**, and displays the results on an **interactive SOC dashboard**.

MGDA: Provenance Graph Threat Detection & Attack Scenario Reconstruction

Citation	Summary	Advantages & Disadvantages
Author: Mengjiao Cui, Zhengwei Jiang, Shuai Li, Chunyan Ma, Kai Zhang, Peian Yang, and Huamin Feng **Year:** 2025 **Journal:** Computer Networks (Elsevier)	Proposes **MGDA**, a self-supervised masked graph autoencoder on system provenance graphs for label-free APT detection. By learning normal behavior through masked node and edge reconstruction, anomalies are flagged via reconstruction errors, followed by causal tracing to reconstruct multi-stage attack scenarios.	**Advantages:** ▪ Detects unknown attacks without needing attack training labels. ▪ Fast training in 14 minutes (100× speedup over baselines). **Disadvantages:** ▪ Connects attack steps using manual rules rather than AI. ▪ Inspects only event links, completely ignoring payload contents.

Commander: Robust Cross-Machine Multi-Phase APT Detection via Provenance

Citation	Summary	Advantages & Disadvantages
Author: Qi Liu, Kaibin Bao, and Veit Hagenmeyer **Year:** 2025 **Journal:** Journal of Information Security and Applications (Elsevier)	Proposes **Commander**, a cross-machine provenance analytics detector that tracks multi-phase APT lateral movement across distributed hosts. It uses logon-session partitioning to bound graph growth and prevent dependency explosion, correlating host audit logs with authentication events to trace attacker progression.	**Advantages:** ▪ Accurately tracks lateral attack movement across multiple hosts. ▪ Session partitioning stops system graphs from growing too large. **Disadvantages:** ▪ Strictly requires Windows Active Directory authentication logs. ▪ Connecting attack paths across machines takes up to 9 minutes.

Threat Detection in CPS Through Spatio-Temporal Anomaly Detection Using GAT

Citation	Summary	Advantages & Disadvantages
Author: Roberto Canonico, Francesco Lista, Annalisa Navarro, Giancarlo Sperlí, and Andrea Vignali **Year:** 2025 **Journal:** Computers & Security (Elsevier)	Proposes an unsupervised spatio-temporal framework combining **Graph Attention Networks (GAT)** for spatial dependencies and recurrent units for temporal patterns. It detects complex cyber-physical anomalies from multivariate sensor streams via joint forecasting and reconstruction errors.	**Advantages:** ▪ Captures both sensor wiring links and rapid timing changes. ▪ Fast 1.4-second detection enables quick real-time response. **Disadvantages:** ▪ Assumes sensor topology stays fixed during operation. ▪ Higher compute overhead compared to traditional baseline models.

EdgeFence: Federated Temporal GNNs for Adversarial Malware Detection in Edge Networks

Citation	Summary	Advantages & Disadvantages
Author: Osei Isaac, Benjamin Appiah, Daniel Commey, Kwabena Owusu-Agyemang, Michael Asante, and Benjamin Hayfrom Acquah **Year:** 2026 **Journal:** International Journal of Information Security (Springer)	Proposes **EdgeFence**, a federated learning framework for lightweight malware detection across distributed edge networks. Edge nodes model system call sequences as dynamic interaction graphs and locally train Temporal GNNs, sharing only model weights via FedAvg to preserve data privacy against adversarial threats.	**Advantages:** ▪ Keeps raw traffic on local devices to protect user privacy. ▪ Temporal graph layers effectively catch evasive malware tricks. **Disadvantages:** ▪ Differential privacy noise slightly lowers overall accuracy. ▪ Constrained by mobile device battery and processing limits.

SHIELD: APT Detection and Intelligent Explanation Using Large Language Models

Citation	Summary	Advantages & Disadvantages
Author: Parth Atulbhai Gandhi, Prasanna N. Wudali, Yonatan Amaru, Akansha Shukla, Yuval Elovici, and Asaf Shabtai **Year:** 2026 **Journal:** Engineering Applications of Artificial Intelligence (Elsevier)	Proposes **SHIELD**, an end-to-end framework combining provenance graph anomaly detection with Large Language Models (LLMs). It isolates suspicious subgraphs from OS audit logs, prompting a fine-tuned LLM to verify attacks, generate plain-English explanations, and map activities to MITRE ATT&CK techniques.	**Advantages:** ▪ Reduces false alarms and analyst workload. ▪ Tracks slow-evolving attacks over long periods. ▪ Provides clear and human-interpretable attack summaries. **Disadvantages:** ▪ Requires high computational resources. ▪ Can cause detection delay. ▪ Depends on complete and accurate log capture.

From Logs to Tactics: Unsupervised APT Reconstruction with MITRE Meta-Alerts

Citation	Summary	Advantages & Disadvantages
Author: Francesco Ferazza, Cosimo Melella, Konstantinos Mersinas, Ricardo Lugo, and Rain Ottis **Year:** 2026 **Journal:** International Journal of Information Security (Springer)	Proposes an unsupervised framework that reconstructs full-scale APT campaigns from noisy audit logs. It uses Graph Autoencoders and Sentence-BERT to generate alert embeddings, clustering them into multi-stage attack sequences and achieving an 87% F1-score in automated MITRE ATT&CK mapping.	**Advantages:** ▪ Reduces alert overload by grouping many alerts into meaningful meta-alerts. ▪ Works without labelled data using an unsupervised approach. ▪ Handles noisy and heterogeneous logs from different sources. **Disadvantages:** ▪ Depends on LLM summary quality, so incorrect summaries can affect later stages. ▪ Ontology-based mapping can miss paraphrased expressions. ▪ Evaluation was mainly based on exercise data, so broader real-world testing is needed.

A Novel Hybrid Intrusion Detection Method Integrating Anomaly and Misuse Detection

Citation	Summary	Advantages & Disadvantages
Author: Gisung Kim, Seungmin Lee, and Sehun Kim **Year:** 2014 **Journal:** Expert Systems with Applications (Elsevier)	Proposes a hierarchical dual-tier intrusion detection architecture integrating misuse and anomaly detection. A frontline Decision Tree rapidly filters known network attack signatures, while decomposed One-Class SVMs profile normal behavior to detect novel zero-day attacks with reduced overhead.	**Advantages:** ▪ Improves detection performance for both known and unknown attacks. ▪ Reduces the training time of the anomaly detection model. ▪ Uses known attack information to build more accurate normal-behavior profiles. **Disadvantages:** ▪ C4.5 may split well-formed normal clusters. ▪ Uneven data distribution reduces the expected reduction in training and testing time. ▪ About 87% of training data was concentrated in one subset, limiting the time-saving benefit.

Self-Supervised Graph Neural Networks for Network Intrusion Detection in Cloud-Edge

Citation	Summary	Advantages & Disadvantages
Author: Haiyang Diao, Xiang Li, Wan Zhang, Yan Sibyl Yu, Ye Wang, and Jing Zhang **Year:** 2026 **Journal:** Journal of Cloud Computing (Springer)	Proposes a self-supervised contrastive Graph Neural Network framework for network intrusion detection in cloud-edge environments. Modeling traffic flows as IP-port communication graphs, it learns robust topological representations from unlabeled traffic via multi-head attention without requiring labeled attack data.	**Advantages:** ▪ Reduces the dependency on labeled data by using self-supervised contrastive learning. ▪ Captures both node and edge information using multi-head attention, which helps represent different malicious traffic patterns. ▪ Reduces computational cost through dynamic attention-based edge pruning, which is useful for resource-constrained edge devices. **Disadvantages:** ▪ High training computational overhead because Wasserstein and Gromov-Wasserstein distances are computationally expensive. ▪ Sensitive to extreme class imbalance, causing poor detection of minority attack classes such as Theft and Backdoor. ▪ Performance varies across datasets, with lower performance on complex datasets.

GraphShield: Advanced Dynamic Graph-Based Malware Detection Using GNNs

Citation	Summary	Advantages & Disadvantages
Author: Eslam Amer, Shaker El-Sappagh, Tamer Abuhamad, Bander Ali Saleh Al-Rimy, and Alaa Mohasseb **Year:** 2025 **Journal:** Expert Systems with Applications (Elsevier)	Proposes **GraphShield**, an explainable malware detection platform that converts dynamic program execution traces into behavioral interaction graphs. An attention-based GNN captures execution dependencies, while GNNExplainer highlights the exact malicious subgraphs to provide actionable explanations for analysts.	**Advantages:** ▪ Detects polymorphic malware by tracking behavioral graphs. ▪ Outperforms static signature scanners on metamorphic variants. **Disadvantages:** ▪ Requires sandbox execution to capture live malware behaviors. ▪ Malware that delays actions or sleeps can evade detection.

Cyber Security Attack Recognition on Cloud Networks Based on GCN and GraphSAGE

Citation	Summary	Advantages & Disadvantages
Author: Fargana Abdullayeva and Suleyman Suleymanzade **Year:** 2024 **Journal:** Results in Control and Optimization (Elsevier)	Proposes modeling cloud network flows as communication graphs to classify multi-class attacks using GCN and GraphSAGE models. The inductive neighborhood sampling of GraphSAGE allows the model to generalize effectively to newly deployed virtual machines without full-graph retraining.	**Advantages:** ▪ Effectively captures communication topology between cloud instances. ▪ GraphSAGE scales well to large, dynamic cloud networks. **Disadvantages:** ▪ Tested on static offline logs rather than live cloud traffic. ▪ Ignores internal host processes running inside virtual machines.

FTG-Net-E: Hierarchical Ensemble Graph Neural Network for DDoS Attack Detection

Citation	Summary	Advantages & Disadvantages
Author: Rana Abu Bakar, Lorenzo De Marinis, Filippo Cugini, and Francesco Paolucci **Year:** 2024 **Journal:** Computer Networks (Elsevier)	Proposes **FTG-Net-E**, a hierarchical ensemble Graph Neural Network for detecting high-volume DDoS floods and volumetric network attacks. By converting traffic into Flow Traffic Graphs (FTG), a two-level ensemble processes local and global traffic topology, achieving 99.2% detection precision.	**Advantages:** ▪ Two-tier design filters massive floods with sub-millisecond delay. ▪ Hierarchical ensemble minimizes false alarms during burst traffic. **Disadvantages:** ▪ Specializes in high-rate DDoS and misses low-and-slow APTs. ▪ Requires hardware acceleration (GPU/FPGA) to maintain wire speed.

Intelligent Intrusion Detection for Cyber-Physical Systems Using GAN-LSTM

Citation	Summary	Advantages & Disadvantages
Author: Md Shakil Siddique, Md. Ashikur Rahman Khan, Ishtiaq Ahammad, Nishu Nath, Joysri Rani Das, and Fardowsi Rahman **Year:** 2025 **Journal:** Franklin Open (Elsevier)	Proposes an intrusion detection framework for cyber-physical systems combining Generative Adversarial Networks (GAN) and LSTM networks. The GAN synthesizes realistic minority-class attack samples to overcome severe data imbalance, while the LSTM captures sequential temporal patterns in industrial telemetry logs.	**Advantages:** ▪ Solves the rare-attack data shortage using synthetic samples. ▪ Successfully improves detection recall for rare intrusions by 28%. **Disadvantages:** ▪ GAN training can become unstable or generate unrealistic sensor values. ▪ Analyzes sensor streams as parallel lines without graph topology.

Knowledge Graph Visualization and Anomaly Detection Using Graph Neural Networks

Citation	Summary	Advantages & Disadvantages
Author: Ali Yilmaz, Resul Das, Mehmet Ozdem, and Berk Canberk **Year:** 2026 **Journal:** Computer Networks (Elsevier)	Represents network communication as a graph, where IP addresses are nodes and communication flows are edges. Applies GCN, GAT, GIN, and GraphSAGE to detect network anomalies and identify suspicious communication patterns. Provides dynamic attack-topology visualization, helping security analysts understand relationships between network entities and possible attack paths.	**Advantages:** ▪ Represents communication between IPs as a graph, making attack connections easier to identify. ▪ GNN models such as GIN and GraphSAGE achieved strong F1 and ROC-AUC results. ▪ Provides attack-topology visualization to help analysts understand attacker–target relationships. **Disadvantages:** ▪ Mainly focuses on network-level graph analysis, not live OS-level provenance. ▪ Does not fully address temporal/dynamic graph behaviour. ▪ Streaming graph updates and low-latency deployment need further development.

Robust, Efficient Detection of Network Intrusion Attacks: A Large Language Model Approach

Citation	Summary	Advantages & Disadvantages
Author: Giancarlo Iannizzotto, Andrea Nucita, and Andrea Siragusa **Year:** 2026 **Journal:** Computer Networks (Elsevier)	Proposes using fine-tuned open-source LLMs (LLaMA-3, Mistral) for contextual network intrusion detection. By serializing tabular flow records into structured text prompts, the language model captures subtle contextual relationships between flow features, demonstrating high resilience against adversarial perturbations.	**Advantages:** ▪ Pre-trained knowledge allows high detection without extensive task tuning. ▪ Shows strong resilience against noisy or slightly altered attack flows. **Disadvantages:** ▪ Token-by-token inference creates significant latency bottlenecks at wire speed. ▪ Requires multi-GPU hardware servers to execute real-time inference.

Detection of Advanced Persistent Threat: A Genetic Programming Approach

Citation	Summary	Advantages & Disadvantages
Author: Abdullah Al Mamun, Harith Al-Sahaf, Ian Welch, and Masood Mansoori **Year:** 2024 **Journal:** Applied Soft Computing (Elsevier)	Proposes a Genetic Programming (GP) feature construction method for detecting multi-stage APT attacks in network traffic. Evolutionary algorithms automatically generate mathematical feature combinations from raw flow metrics, separating low-and-slow malicious connections from routine background traffic.	**Advantages:** ▪ Automatically discovers non-linear feature combinations without expert manual rules. ▪ Evolved decision trees remain lightweight and interpretable for human inspection. **Disadvantages:** ▪ Evolutionary search requires extensive multi-generation compute time. ▪ Operates solely on static flow totals, missing host causal dependencies.

Dynamic Cyber Deception Using AI-Driven Adaptive Honeypot Networks

Citation	Summary	Advantages & Disadvantages
Author: Mohammed Alshehri **Year:** 2026 **Journal:** Ain Shams Engineering Journal (Elsevier)	Proposes an AI-driven dynamic cyber deception platform that deploys adaptive honeypot networks against multi-stage APT campaigns. Using reinforcement learning, it continuously reconfigures decoy services and synthetic vulnerabilities based on attacker probing, diverting adversaries and capturing high-fidelity telemetry.	**Advantages:** ▪ Dynamically changes honeypot behaviour according to attacker activity. ▪ Makes deception more difficult for attackers to identify. ▪ Records attacker activities and helps identify their tactics and techniques. **Disadvantages:** ▪ Effectiveness depends on attackers interacting with the honeypot environment. ▪ Continuous changes may cause over-reactive or unrealistic behaviour.

5. Identified Research Gaps & Proposed Solutions

Research Gaps (Existing Systems)

- **1. Isolated Logs and Missed Attack Paths:**
  Existing tools analyze each computer separately, making it difficult to track attackers moving between machines. Combining all logs can also increase storage and processing costs.
- **2. Dependence on Known Attacks:**
  Many AI models require labeled attack data, making it difficult to detect new, unknown zero-day attacks and increasing false alarms.
- **3. Privacy Risks and High Resource Usage:**
  Sharing raw logs creates privacy risks, while complex graph AI models require too much time and computing power for resource-limited devices.

Proposed Solutions (Our System)

- **1. Network-Wide Attack Tracking:**
  Connect related events across multiple computers to reconstruct complete attack paths while reducing unnecessary graph complexity.
- **2. Unsupervised Zero-Day Detection:**
  Use Masked-GAT to learn normal system behavior and identify unknown attacks without requiring labeled attack data.
- **3. Lightweight and Privacy-Preserving AI:**
  Run a compact model locally for fast detection and use TenSEAL CKKS-encrypted Federated Learning to train collaboratively without sharing raw system logs.

1. Core Architectural Pillars of AEGIS-PROV

- **Continuous Multi-OS Audit Ingestion:**
  Passively captures low-level operating system events in real time from Linux kernel audit hooks, Windows Sysmon, and industrial control streams, converting them into uniform graph records.
- **Semantics-Preserving Event Compaction:**
  Intelligently merges high-frequency repetitive operations (such as continuous file polling and socket reads), collapsing log volume by $> 85\%$ without losing any causal attack steps.
- **Decoupled Feature Representation:**
  Converts graph entities into lightweight 6D numeric vectors for microsecond GPU processing, while offloading long textual strings (file paths, hashes, IP addresses) into external lookup tables.
- **Dual-Tier AI Detection Engine:**
  Combines an on-device Edge-TGNN (< 80 KB, 3.2 ms) for rapid local containment with an enterprise-level Masked-GAT autoencoder that learns normal behavior to catch unknown zero-day attacks.
- **Whole-Network Causal Tracing:**
  Uses logon-session boundaries and synthetic pseudo-edges to trace multi-stage lateral movement across Windows AD, Linux, and OT networks without overwhelming system memory.

2. Technologies, Frameworks & Core Libraries

- **Deep Learning & Graph Frameworks:**
  Built with **Python 3.10+**, **PyTorch v2.2.0**, **PyTorch Geometric (PyG v2.5.0)**, and **DGL v1.1** for high-performance temporal graph modeling and spatial message passing.
- **Privacy-Preserving Federated Cryptography:**
  Utilizes **TenSEAL CKKS Homomorphic Encryption** for secure federated weight aggregation, allowing multiple organizations to train collaboratively without sharing raw logs or gradients.
- **High-Speed Ingestion & Spatial Mathematics:**
  FastAvro v1.9 for high-throughput binary CDM parsing, **SciPy v1.12** for high-dimensional K-d Tree spatial anomaly indexing ($k = 10$), **NumPy**, and **Scikit-Learn**.
- **Kernel Sensors & Probes:**
  Linux Auditd, **CamFlow v0.5** Linux Security Module (LSM) hooks, and **Windows Sysmon v14** with Windows Event Forwarding (WEF).
- **Forensic Dashboard & Microservices:**
  Asynchronous **FastAPI** REST backend coupled with a **Cytoscape.js** interactive graph canvas for real-time SOC kill-chain investigation.

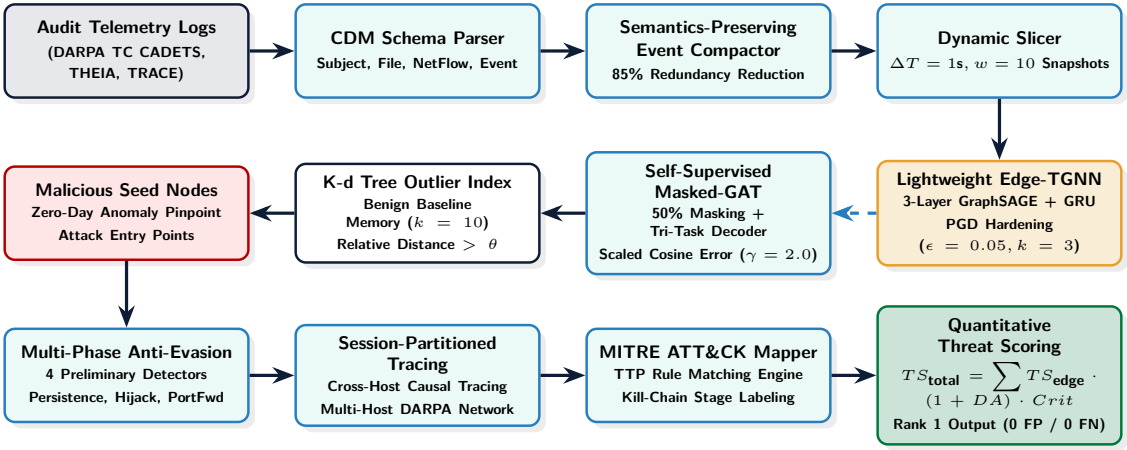
3. Benchmark Dataset: DARPA Transparent Computing (TC)

- **Gold Standard for APT Provenance Analytics:**
  The project exclusively utilizes the **DARPA Transparent Computing (TC) Engagement 3 (E3)** dataset, the authoritative benchmark for evaluating enterprise-wide provenance-based intrusion detection systems.
- **Subsets Evaluated (Real Multi-Stage Red-Team Attacks):**
  - **E3-CADETS (FreeBSD / OpenBSM):** 1.62M nodes, 3.30M edges. Evaluates detection of an Nginx memory corruption exploit, spawned shell, payload dropper (`/tmp/vUgefal`), and in-memory injection into `sshd`.
  - **E3-THEIA (Linux Ubuntu / LSM):** 1.62M nodes, 2.87M edges. Evaluates defense against browser drive-by exploits, root privilege escalation, credential theft, and sensitive file exfiltration.
  - **E3-TRACE (Linux Ubuntu / CamFlow & Auditd):** 3.28M nodes, 4.08M edges. Continuous 2-week recording featuring a stealthy Firefox backdoor, cron persistence, and SSH lateral movement across machines.
- **Why Exclusively DARPA TC? Ground-Truth Enterprise Provenance:**
  Contains authentic multi-stage cyber attacks executed by nation-state caliber red teams interleaved with dense benign background activity. It provides full kernel-level causal visibility across processes, files, and network sockets under a unified Common Data Model (CDM).

4. Dual-Tier Database Architecture: Graph + Relational

- **Neo4j Enterprise 5.x (Graph Provenance Store):**
  Purpose-built for relationship traversal. Stores whole-system provenance DAGs with indexed node entities (`:Process`, `:File`, `:Socket`, `:Host`) and causal relationship edges (`[:FORK]`, `[:READ]`, `[:WRITE]`, `[:CONNECT]`, and synthetic pseudo-edges); native graph index pointers deliver sub-second multi-hop causal queries.
- **PostgreSQL 16 (Relational Metadata Store):**
  Guarantees ACID compliance for structured security records:
  - `hosts`: Tracks connected endpoint asset inventory, platform details, and health status.
  - `anomalies`: Logs individual AI anomaly detection seeds, scores, and detector sources.
  - `attack_graphs`: Stores reconstructed multi-stage attack DAGs with MITRE ATT&CK tags.
  - `federated_updates`: Records model weights, loss deltas, and training rounds.

7. Workflow Diagram: End-to-End System Processing Pipeline



8. Proposed Methods: Lightweight Edge-TGNN with Adversarial Defense

1. Lightweight Edge-TGNN with On-Device PGD Hardening (Edge Sentry)

- **Spatial GNN (3-Layer GraphSAGE Encoder):**
  Gathers causal context from immediate process-file-socket neighborhoods with hidden size $d_h = 32$, ensuring an ultra-low memory footprint (< 1 MB RAM):

$$h_v^{(l)} = \text{ReLU}\left(W_{\text{self}}^{(l)} h_v^{(l-1)} + W_{\text{neigh}}^{(l)} \frac{1}{|N(v)|} \sum_{u \in N(v)} h_u^{(l-1)} \right)$$

- **Temporal Recurrence (Gated Recurrent Unit):**
  Captures multi-second temporal execution sequences across rolling snapshots ($\Delta T = 1\text{s}, w = 10, d_{\text{gru}} = 64$):

$$h_{i,t}^{\text{temp}} = \text{GRU}(h_{G_t}, h_{i,t-1}^{\text{temp}})$$

- **3-Step Continuous PGD Adversarial Hardening:**
  Simulates evasive attacker noise (dummy syscall injection, $k_{\text{mod}} \leq 3, \epsilon = 0.05$) directly on-device during training, hardening the model against evasion before deployment:

$$\delta^* = \arg \max_{\|\delta\| \leq \Omega} \ell(f_\theta(G + \delta), y)$$

- **Privacy-Preserving Federated Collaboration:**
  Transmits only 79.63 KB model parameter deltas $\Delta\theta_i$ protected by TenSEAL CKKS encryption; sensitive raw enterprise telemetry never leaves the endpoint.

2. Self-Supervised Masked-GAT Autoencoder (Zero-Day Pinpointing)

- **50% Stochastic Attribute Masking:**
  Hides half of the node features while preserving structural graph causality, forcing the model to deeply learn normal everyday system behavior without requiring labeled attack examples.
- **Multi-Layer Attention Message Passing:**
  Computes edge-type aware attention coefficients $\alpha_{nu}^{(l)}$ and concatenates multi-scale contextual representations across all layers: $H_n = h_n^{(1)} \parallel h_n^{(2)} \parallel h_n^{(3)}$.
- **Tri-Task Self-Supervised Decoder:**
  Optimizes three joint objectives simultaneously:
  - *Task 1 (Feature Reconstruction)*: Scaled Cosine Error ($\gamma = 2.0$) focusing on hard-to-predict anomalies.
  - *Task 2 (Latent Target Matching)*: Aligns corrupted representations with an exponential moving average target.
  - *Task 3 (Structure Link Prediction)*: Reconstructs causal edges via Binary Cross Entropy.
- **High-Dimensional K-d Tree Outlier Scoring:**
  Compares test node representations against the 10 nearest normal neighbors in benign memory ($k = 10$); nodes exceeding threshold $\theta = 1.5$ are flagged as anomalous zero-day attack seeds $\mathcal{M}$.

3. Session-Partitioned Cross-Machine Tracing Engine (Commander Core)

- **Logon-Session Boundary Graph Partitioning:**
  Naive graph traversal causes catastrophic *dependency explosion*. By partitioning whole-system logs into user Logon Sessions (L_{id}), reachability is tightly bounded, preventing traversal bloat.
- **Synthetic Pseudo-Edge Resolution:**
  Disjoint attack steps separated by execution boundaries are bridged by synthetic links:
  - *Persistence*: Links registry Run keys and scheduled task setups to their later trigger execution.
  - *Session Hijacking*: Links token duplication and elevation to new unauthorized logon sessions.
  - *Port Forwarding*: Remaps local proxy loopback sockets back to remote attacker command IPs.
- **Enterprise Multi-Host Lateral Correlation:**
  Seamlessly follows lateral pivot chains across enterprise workstations and server endpoints (FreeBSD and Linux) within the DARPA multi-host network.

4. Quantitative ATT&CK Threat Scoring & Incident Prioritization

- **Edge Threat Score Formulation:**
  Every causal transition edge e in the reconstructed attack graph is evaluated based on tactic severity and frequency:
  $$TS_{\text{edge}} = \sum_i w_i \cdot [Freq(teq_i) + Var(tac_i)]$$
  where credential theft ($w = 3.0$) and persistence ($w = 2.5$) carry the highest weights.
- **Total Scenario Threat Score (TS_{total}):**
  Aggregates across all lateral movement edges with critical asset multipliers:
  $$TS_{\text{total}} = \sum TS_{\text{edge}} \cdot (1 + \mathbb{I}_{\text{DomainAdmin}}) \cdot \text{Criticality}_{\text{host}}$$
- **Rank-1 Incident Prioritization (Zero False Alarms):**
  Scores and sorts all candidate subgraphs, successfully surfacing the true multi-stage APT campaign at **Rank 1 (0 FP / 0 FN)** on the interactive SOC dashboard.
- **MITRE ATT&CK Mapping:**
  Automatically maps every step of the reconstructed graph to official MITRE tactics (Initial Access → Execution → Persistence → Lateral Movement → Exfiltration).

9. Model Responsibilities: Model 1 — Edge-TGNN (Perimeter Sentry)

Model 1: Lightweight Edge-TGNN (On-Device Perimeter Sentry)

- **Primary Operational Role:**
  First line of defense deployed on local edge nodes, IoT gateways, and endpoint agents for millisecond-level screening.
- **Input Data Stream:**
  Dynamic rolling temporal graph snapshots $\{G_{t-9}, \ldots, G_t\}$ with slice duration $\Delta T = 1$s and window $w = 10$.
- **Neural Architecture:**
  3-Layer GraphSAGE (mean aggregation, $d_h = 32$) coupled to a 1-Layer GRU ($d_{\text{gru}} = 64$) and global mean pooling readout.
- **Footprint & Latency:**
  Ultra-compact 79.63 KB parameter footprint, requires < 1 MB RAM, and executes inference in just **3.2 ms** per snapshot.
- **Adversarial Defense:**
  Hardened against structural evasion via an on-device 3-step continuous PGD loop ($\epsilon = 0.05, k_{\text{mod}} \leq 3$).
- **Detection Criterion & Output:**
  Calculates local maliciousness probability score $s_{i,t} \in [0, 1]$. Dispatches an immediate edge isolation alarm and triggers SOC escalation whenever $s_{i,t} \geq \tau_{\text{edge}}$.

9. Model Responsibilities: Model 2 — Masked-GAT (SOC Deep Analyzer)

Model 2: Self-Supervised Masked-GAT Autoencoder (SOC Deep Analyzer)

- **Primary Operational Role:**
  Centralized enterprise analyzer running on GPU clusters to pinpoint zero-day process execution anomalies without labeled training data.
- **Input Data Stream:**
  Compacted whole-host provenance graphs generated from DARPA TC audit streams with repetitive loops collapsed ($> 85\%$ compaction).
- **Neural Architecture:**
  Stacked multi-layer GAT encoder with edge-type attention and layer concatenation, paired with a Tri-Task decoder.
- **Self-Supervised Masking & Loss:**
  Applies 50% stochastic attribute masking. Optimized using Scaled Cosine Error ($\gamma = 2.0$) to focus reconstruction on subtle anomalies.
- **Detection Criterion (Spatial Outlier Scoring):**
  Embeds reconstructed vectors into a benign memory indexed by a K-d tree ($k = 10$). Process nodes with relative spatial distance $\frac{\bar{D}_{\text{test}}}{\bar{D}_{\text{train}}} > \theta$ ($\theta = 1.5$) are flagged as anomalous seed processes $\mathcal{M}$.

Model 3: Whole-Network Tracing Engine (Causal Kill-Chain Reconstructor)

- **Primary Operational Role:**
  Connects disconnected attack steps across execution boundaries, machines, and networks to reconstruct the full multi-phase attack story.
- **Input Data Sources:**
  AI anomaly seed processes $\mathcal{M}$ from Masked-GAT + enterprise host authentication logs (SSH / Kerberos / Logons).
- **Anti-Evasion Pseudo-Edge Resolver:**
  Inserts synthetic causal edges to bridge:
  - *Persistence*: Connects startup registry / scheduled task creation to subsequent execution.
  - *Session Hijacking*: Connects `SYSTEM` token duplication to unauthorized logon sessions.
  - *Port Forwarding*: Remaps local proxy loopback sockets to remote target endpoints.
- **Attribution & Output:**
  Applies quantitative threat scoring TS_{total}, maps attack stages to the MITRE ATT&CK framework, and displays ranked attack DAGs at **Rank 1 (0 FP / 0 FN)** on the interactive SOC dashboard.

10. Inference Flow: Phase 1 — Real-Time Edge Screening

Phase 1: Real-Time Edge Screening & Seed Isolation

1. **Step 1 — Local Edge Screening & Anomaly Flagging:**
   Streaming OS audit events are ingested and transformed into dynamic temporal snapshots ($\Delta T = 1\text{s}, w = 10$). The lightweight Edge-TGNN evaluates each snapshot in just 3.2 ms on local edge devices. If the maliciousness score $s_{i,t} > \tau_{\text{edge}}$, an immediate local isolation alert is dispatched and compacted telemetry is forwarded to the SOC cluster.

2. **Step 2 — Self-Supervised Zero-Day Pinpointing:**
   The centralized Masked-GAT autoencoder reconstructs masked node features across the candidate subgraph. A high-dimensional spatial K-d tree evaluates reconstruction discrepancy against normal baseline vectors, isolating precise attack seed process nodes $\mathcal{M}$ with zero labeled training data.

10. Inference Flow: Phase 2 — Whole-Network Tracing

Phase 2: Whole-Network Tracing & Threat Prioritization

3. **Step 3 — Multi-Host Causal Kill-Chain Tracing:**
   Starting from seed processes $\mathcal{M}$, the tracing engine initiates bidirectional BFS bounded by Logon Sessions. Synthetic pseudo-edges stitch together persistence mechanisms, stolen credentials, and reverse proxy port forwards across Active Directory, Linux jump servers, and OT controllers.

4. **Step 4 — Final Output Generation & Threat Ranking:**
   Reconstructed DAGs are automatically mapped to MITRE ATT&CK tactics and scored by the quantitative formula TS_{total}. The true multi-stage APT scenario is presented at **Rank 1 (0 FP / 0 FN)** on the Cytoscape.js SOC dashboard.

1. DARPA Transparent Computing (TC) Program Background

- **Enterprise Scale Realism:**
  Conducted by DARPA to advance whole-system provenance tracing across diverse operating systems and distributed enterprise networks.
- **Authentic Adversarial Red-Team Campaigns:**
  Contains authentic multi-stage cyber attacks executed by nation-state caliber red teams interleaved with dense benign background activity.
- **Standardized Common Data Model (CDM v18/v20):**
  - `Subject`: Process nodes with PID, PPID, command line, and user credentials.
  - `FileObject`: File nodes with absolute path, size, permissions, and cryptographic hash.
  - `NetFlow`: Network sockets with local/remote IP addresses and port numbers.
  - `Event`: Directed edges (`EXEC`, `FORK`, `READ`, `WRITE`, `CONNECT`) with nanosecond timestamps.

2. Benchmark Subsets Investigated

- **E3-CADETS (FreeBSD 11.1 / OpenBSM):**
  1.62M nodes, 3.30M edges. Ground truth attacks include Nginx HTTP memory corruption exploit, dropper installation, and Drakon in-memory injection into `sshd`.
- **E3-THEIA (Linux Ubuntu / LSM Probes):**
  1.62M nodes, 2.87M edges. Ground truth attacks include Firefox browser drive-by download, local root privilege escalation, and sensitive credential exfiltration.
- **E3-TRACE (Linux Ubuntu / CamFlow & Auditd):**
  3.28M nodes, 4.08M edges. Continuous 2-week recording cycle with Firefox backdoor installation, persistent cron modification, and lateral SSH pivot.

1. Semantics-Preserving Event Compaction

- **Loop Collapse Mechanism:**
  Eliminates repetitive high-frequency audit noise (e.g. repeated file cache reads, continuous socket polling):

$$\text{If } e_1 = (s, r, o, t_1) \text{ and } e_2 = (s, r, o, t_2) \text{ with same state,}$$

  merge into e_1 and increment edge frequency weight:

$$w(e) \leftarrow w(e) + 1$$

- **Exceptional Compression Performance:**
  Reduces total audit graph volume by $> 85\%$ while preserving 100% of causal dependencies and attack paths.
- **Zero Causality Loss Guarantee:**
  Proven to maintain exact reachability for forward/backward causal tracking.

2. Decoupled Feature Representation

- **Compact 6D One-Hot Embedding:**
  Generates a lightweight vector $\mathbf{X}_v \in \{0,1\}^6$ encoding:
  - Entity Type (Process, File, Socket, Controller)
  - Operational Role & System Privilege Level
- **Offloaded String Lookup Tables:**
  High-overhead variable-length strings (absolute file paths, SHA256 hashes, command-line arguments, IP addresses) are decoupled into an external key-value store.
- **High Computational Efficiency:**
  Keeps node tensor representations fixed-size, enabling ultra-fast batch GPU tensor multiplications in GNN layers.

12. Model Training: Chronological Data Splitting Strategy

1. Temporal Order Guarantee

- **Prevention of Data Leakage:**
  Standard random k-fold cross-validation causes future-to-past data leakage in provenance graphs. We strictly partition audit logs chronologically along the timeline.
- **Realistic Operational Setup:**
  Simulates genuine SOC deployment where models are trained exclusively on historical benign activity and evaluated on future unseen enterprise traffic.
- **Timeline Alignment:**
  Preserves inter-event time deltas $\Delta t = t_j - t_i$ across snapshot boundaries.

2. Partition Splits & Anomaly Threshold Calibration

- **Training Split (Earliest 70%):**
  Consists of 100% benign background execution logs. Used to train the self-supervised Masked-GAT autoencoder and build the K-d tree normal baseline memory.
- **Validation Split (Middle 10%):**
  Benign logs used to calibrate the anomaly threshold $\theta = 1.5$, strictly guaranteeing a false positive rate FPR $\leq 0.1\%$.
- **Testing Split (Latest 20%):**
  Unseen timeline containing actual red-team APT attacks (Nginx exploit, Drakon injection) mixed with background traffic to evaluate detection and reconstruction accuracy.

1. Optimization Objectives & Loss Formulations

- **Adversarial Training Loss (Edge-TGNN):**
  Minimizes empirical risk on clean and PGD-perturbed graphs:

$$\mathcal{L}_{\text{adv}}(\theta) = \text{BCE}(f_\theta(G), y) + 0.8 \cdot \text{BCE}(f_\theta(G + \delta^*), y)$$

  where δ^* is generated via 3-step continuous PGD ascent with $\epsilon = 0.05$.

- **Tri-Task Self-Supervised Loss (Masked-GAT):**
  Combined multi-task objective:

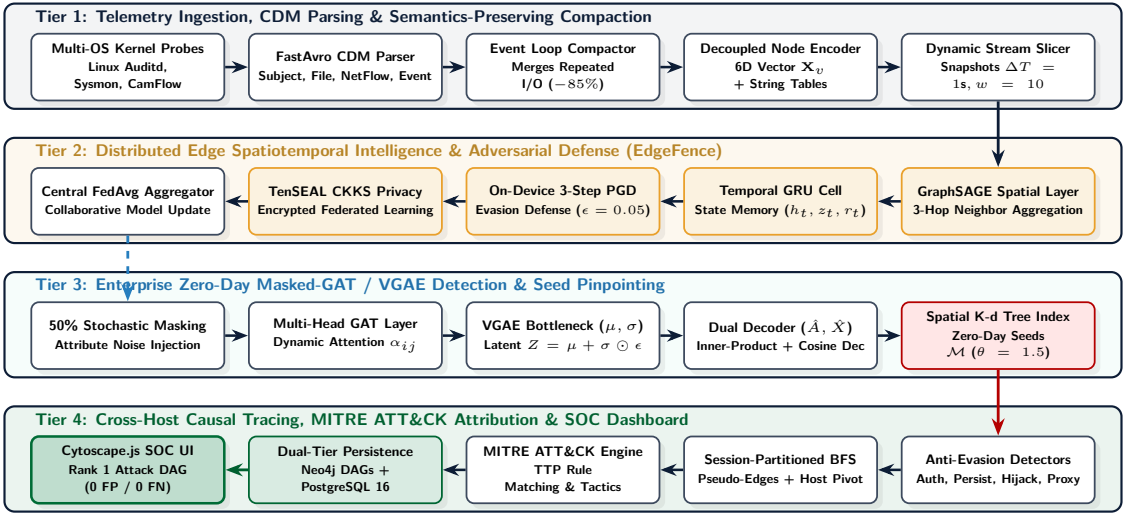
$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{input}} + \mathcal{L}_{\text{latent}} + 0.5\mathcal{L}_{\text{struct}}$$

  utilizing Scaled Cosine Error with scaling exponent $\gamma = 2.0$ to emphasize hard-to-reconstruct nodes.

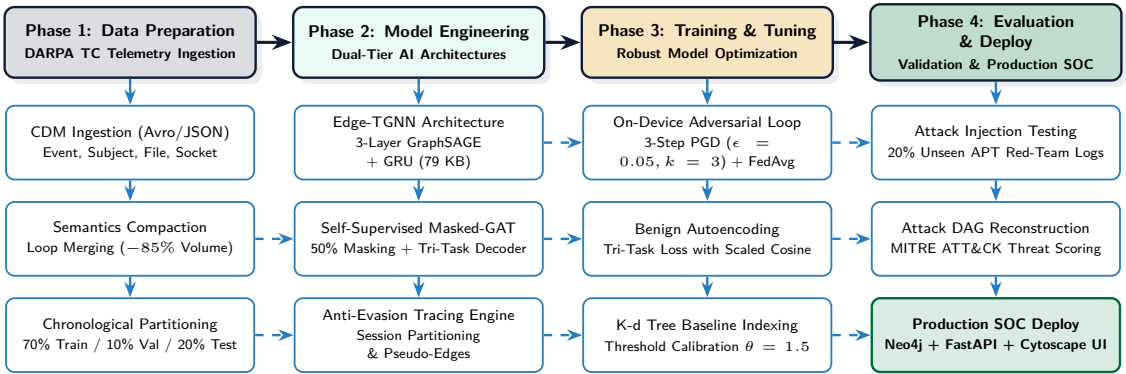
2. Comprehensive Model Evaluation Metrics

- **Classification Performance:**
  Precision, Recall, F1-Score, and False Positive Rate (FPR), ensuring near-zero false alarms ($\leq 0.22\%$).
- **Operational Efficiency:**
  Parameter memory footprint (79.63 KB for Edge-TGNN) and per-snapshot inference latency (< 4 ms).
- **Attribution Accuracy:**
  Scenario Reconstruction Accuracy measured by whether the true attack DAG is ranked at Top-1 (TS_{total} Rank 1 with 0 FP / 0 FN).
- **Adversarial Robustness:**
  Accuracy retention under structural graph evasion attacks ($< 1.8\%$ drop under PGD).

13. System Architecture Diagram: Enterprise-to-Edge Threat Detection



14. Project Development and Testing Flowchart



15. Database Schema: Neo4j Graph Database Property Schema

1. Neo4j Graph Database Property Schema (Whole-System Provenance)

Node Labels & Key Properties:

- `:Process`: {uuid, pid, name, path, cmdline, uid, session_id, host}
- `:File`: {uuid, path, name, file_type, size_bytes, permissions, hash}
- `:Socket`: {uuid, src_ip, src_port, dst_ip, dst_port, protocol}
- `:Controller`: {uuid, model, ip, protocol, firmware, trip_curve}

Directed Relationship Types & Causal Edges:

- `[:FORK | :EXEC]`: (Process) → (Process/File) {timestamp}
- `[:READ | :WRITE]`: (Process) → (File) {timestamp, count}
- `[:CONNECT]`: (Process) → (Socket) {timestamp, state}
- `[:PERSISTENCE_PSEUDO_EDGE]`: (SetupSession) → (ExecSession)
- `[:PORTFWD_CROSS_MACHINE]`: (ProxySrc) → (TargetDst)

2. Relational Metadata Store (PostgreSQL 16 Tables)

- `hosts` **(Endpoint Inventory):**
  Tracks connected infrastructure with schema:
  - `host_id` (PK), `hostname`, `os_platform`, `domain_name`, `ip_addr`, `is_dc`, `last_seen`
- `anomalies` **(AI-Flagged Seed Events):**
  Persists detected anomaly seeds from Edge-TGNN and Masked-GAT:
  - `anomaly_id` (PK), `host_id` (FK), `session_id`, `score`, `detector_source`, `timestamp`
- `attack_graphs` **(Reconstructed Scenarios):**
  Stores causal DAGs with complete MITRE ATT&CK mapping:
  - `graph_id` (PK), `seed_uuid`, `threat_score`, `tactics_list`, `node_count`, `edge_count`, `graph_json`
- `federated_updates` **(Collaborative Training Logs):**
  Records parameter norms and loss deltas across federated aggregation rounds:
  - `round_id` (PK), `client_id`, `loss_clean`, `loss_adv`, `param_norm`, `timestamp`

16. Master Algorithm: Unified End-to-End Threat Detection Pipeline

Master Pipeline: Orchestrating All 5 Project Working Phases

Intuition: Synchronizes data ingestion, event loop compaction, real-time edge screening, central zero-day pinpointing, and cross-machine causal tracing into a unified defense pipeline.

Master Algorithm: Complete Threat Detection & Scenario Reconstruction
Input: Raw Audit Telemetry Stream $\mathcal{S}_{\text{audit}}$, Auth Logs $\mathcal{A}$, Baseline Profile $\mathcal{D}_{\text{norm}}$, TTP Rules $\mathcal{R}$.
Output: Ranked list of prioritized multi-host Attack Scenario DAGs $\mathcal{L}_{\langle AG,TS\rangle}$.

// Phase 1: Ingestion & Semantics-Preserving Compaction
Ingest CDM records (Subject, File, NetFlow, Event); Collapse repetitive loops: $w(e) \leftarrow w(e) + 1$ (-85% volume).
Slice stream into rolling snapshot graph windows $G_{i,t,w} = \{G_{t-9}, \ldots, G_t\}$ with $\Delta T = 1.0$ s, $w = 10$.
// Phase 2: First-Line Edge Anomaly Screening (Algorithm 1: EdgeFence)
Forward $G_{i,t,w}$ through on-device Edge-TGNN: $s_{i,t} \leftarrow \sigma(W_{\text{out}}\text{GRU}(h_{G_i}) + b)$ in < 4 ms.
if $s_{i,t} > \tau_{\text{edge}}$ then Dispatch high-priority alert; Transmit compacted subgraph G_{anom} to SOC cluster.
// Phase 3: Self-Supervised Zero-Day Seed Detection (Algorithm 2: MGDA)
Pass G_{anom} into Masked-GAT autoencoder; Reconstruct 50% masked node attributes: $\hat{X} \leftarrow f_D(f_E(\tilde{X}, E))$.
Query K-d tree normal baseline memory ($k = 10$); Compute normalized outlier score: $\text{Score}(v) = \frac{\bar{D}_{\text{test}}(h_v)}{\bar{D}_{\text{train}}}$.
Extract set of malicious seed process entities: $\mathcal{M} \leftarrow \{v \mid \text{Score}(v) \geq \theta \text{ and type}(v) == \text{Process}\}$.
// Phase 4: Cross-Machine Tracing & Pseudo-Edge Resolution (Algorithm 3: Commander)
Run 4 preliminary detectors: Authentication anomaly, persistence, session hijack, and port forwarding proxy.
for each seed $v \in \mathcal{M}$ do: Identify LogonSessionID; Insert pseudo-edges; Perform bidirectional BFS.
if gateway / jump-server session is traversed then Trace across IT domain into OT industrial controller logs.
// Phase 5: MITRE ATT&CK Annotation and Threat Score Calculation
Match graph paths against TTP rules $\mathcal{R}$; Annotate tactics; Compute $TS_{\text{total}} = \sum TS_{\text{edge}} \cdot (1 + DA) \cdot Crit.$
return Ranked list $\mathcal{L}_{\langle AG,TS\rangle}$ sorted descending by TS_{total} (Surfacing true attack at Rank 1).

16. Algorithm 1: On-Device Federated Adversarial TGNN Training

Phase I Algorithm: On-Device Edge-TGNN Training with Adversarial Hardening (EdgeFence)

Intuition: Protects edge models against attacker evasion tricks (dummy syscall additions) via 3-step PGD, while TenSEAL CKKS encrypted FedAvg preserves employee data privacy.

Algorithm 1: Federated Adversarial TGNN Training

Input: Clients $\mathcal{S} = \{1, \ldots, N\}$, Local datasets $\mathcal{D}_i$, Rounds $R = 100$, Local Epochs $E = 1$, Learning rate $\eta = 0.001$, PGD steps $K_{\text{att}} = 3$, Bound $\epsilon = 0.05$, Edge budget $k_{\text{mod}} = 3$, Adv weight $\lambda = 0.8$.

Output: Adversarially Hardened Global Model $\theta^{(R)}$ (< 80 KB).

Central Server initializes global model $\theta^{(0)}$ (~ 79.63 KB).

for each federated round $r = 1, \ldots, R$ do:

  Server selects client subset $\mathcal{S}^{(r)} \subseteq \{1, \ldots, N\}$ of size $S_{\text{rate}} \cdot N$ ($S_{\text{rate}} = 0.1$).

  Server broadcasts current global parameters $\theta^{(r)}$ to selected clients.

  for each client $i \in \mathcal{S}^{(r)}$ in parallel do:

    Initialize local parameters $\theta_i \leftarrow \theta^{(r)}$.

    for each local epoch $e = 1$ to E do:

      for each mini-batch $(G_T, y) \in \mathcal{D}_i$ do:

        // 1. On-Device PGD Adversarial Perturbation Generation:

        Initialize $\delta \leftarrow 0$; for step $= 1$ to K_{att} do: $\delta \leftarrow \text{clip}\Big(\delta + \alpha \cdot \text{sgn}(\nabla_\delta \ell(f_{\theta_i}(G_T + \delta), y)), -\epsilon, \epsilon\Big)$; Project discrete edges $\leq k_{\text{mod}}$.

        Generate adversarial graph sequence: $G'_T \leftarrow G_T + \delta^*$.

        // 2. Composite Loss Minimization:

        $\mathcal{L}_i^{\text{adv}} = \text{BCE}(f_{\theta_i}(G_T), y) + \lambda \cdot \text{BCE}(f_{\theta_i}(G'_T), y)$.

        Update weights: $\theta_i \leftarrow \theta_i - \eta \nabla_{\theta_i} \mathcal{L}_i^{\text{adv}}$ using Adam optimizer.

    Compute parameter delta: $\Delta\theta_i^{(r)} \leftarrow \theta_i - \theta^{(r)}$; Upload $\Delta\theta_i^{(r)}$ via TenSEAL CKKS to Server.

  Server performs FedAvg aggregation: $\theta^{(r+1)} \leftarrow \theta^{(r)} + \sum_{i \in \mathcal{S}^{(r)}} \frac{|\mathcal{D}_i|}{\sum_j |\mathcal{D}_j|} \Delta\theta_i^{(r)}$.

return Final Hardened Global Model $\theta^{(R)}$.

16. Algorithm 2: Self-Supervised Masked-GAT Autoencoder & Outlier Detection

Phase II Algorithm: Self-Supervised Masked-GAT & K-d Tree Outlier Detection (MGDA)

Intuition: Learns normal system baseline behavior from 100% benign execution logs; isolates unknown zero-day attack seeds when reconstruction distance exceeds the normal K-d tree neighborhood.

Algorithm 2: Self-Supervised Training and K-d Tree Outlier Detection

Input: Compressed Provenance Graph $G = (V, E, X)$, Mask Ratio $\rho = 0.5$, Re-mask Views $K = 3$, Scaling Exponent $\gamma = 2.0$, Neighbors $k = 10$, Outlier Threshold $\theta = 1.5$.

Output: Set of Malicious Seed Process Entities $\mathcal{M}$.

// Phase A: Self-Supervised Training on Benign Baseline Graphs:

Initialize GAT Encoder f_E, Feature Decoder f_D, Latent Projector g, Edge Predictor MLP, Target Generator f_T.

for epoch $= 1$ to E_{train} **do:**

  Randomly sample 50% nodes $\tilde{V} \subset V$; Replace features: $\tilde{X}_i \leftarrow [MASK]$ for $v_i \in \tilde{V}$.

  Forward corrupted graph through GAT: $H \leftarrow f_E(\tilde{X}, E) = h^{(1)} \parallel h^{(2)} \parallel h^{(3)}$.

  // Task 1: Multi-view random re-mask decoding:

  Compute $\mathcal{L}_{\text{input}} = \frac{1}{K} \sum_{k=1}^{K} \sum_{v_i \in \tilde{V}} (1 - \cos(x_i, f_D(\tilde{H}^{(k)})))^{\gamma}$.

  // Task 2: Latent target representation prediction:

  Compute target $Z \leftarrow f_T(X, E)$; Compute $\mathcal{L}_{\text{latent}} = \sum_{v_i \in \tilde{V}} (1 - \cos(z_i, g(H_i)))^{\gamma}$.

// Task 3: Structure Edge Prediction:

Predict positive edges E^+ and negative pairs E^-: $\mathcal{L}_{\text{struct}} = \text{BCE}(\text{MLP}([H_u \parallel H_v]), Y_{\text{edge}})$.

Optimize total loss: $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{input}} + \mathcal{L}_{\text{latent}} + 0.5\mathcal{L}_{\text{struct}}$ via Adam.

// Phase B: K-d Tree Indexing and Anomaly Scoring:

Extract all benign node representations H_{train}; Index into K-d Tree $\mathcal{T}$.

for each entity embedding $h_v \in H_{\text{test}}$ **do:**

  Query $\mathcal{T}$ for $k = 10$ nearest neighbors; Compute normalized outlier score: $\text{Score}(v) = \frac{\bar{D}_{\text{test}}(h_v)}{\bar{D}_{\text{train_neighbors}}}$.

  if $\text{Score}(v) \geq \theta$ **and** $\text{type}(v) ==$ Process **then** $\mathcal{M} \leftarrow \mathcal{M} \cup \{v\}$.

return Malicious Seed Nodes $\mathcal{M}$.

16. Algorithm 3: Cross-Machine Attack Graph Tracing & Scenario Reconstruction

Phase III Algorithm: Cross-Machine Tracing & MITRE Attribution (Commander)

Intuition: Bounded bidirectional BFS tracks multi-host attacker movement without graph explosion; pseudo-edges connect persistence setups, token duplication, and proxy tunnels.

Algorithm 3: Robust Whole-Network Tracing and ATT&CK Reconstruction
Input: Audit events $\mathcal{E}$, Authentication logs $\mathcal{A}$, Malicious Seeds $\mathcal{M}$, TTP Rules $\mathcal{R}_{\text{Host}}, \mathcal{R}_{\text{Net}}$.
Output: Ranked list of Attack Graphs and Threat Scores $\mathcal{L}_{\langle AG,TS\rangle}$.

Run 4 Preliminary Detectors in parallel:
  $\mathcal{L}_{\text{auth}} \leftarrow$ DetectAuthenticationAnomaly($\mathcal{A}$); $\mathcal{L}_{\text{pers}} \leftarrow$ DetectPersistence($\mathcal{E}$);
  $\mathcal{L}_{\text{hijack}} \leftarrow$ DetectSessionHijacking($\mathcal{E}, \mathcal{A}$); $\mathcal{L}_{\text{portfwd}} \leftarrow$ DetectPortForwarding($\mathcal{E}, \mathcal{A}$).
for each anomaly in $\mathcal{L}_{\text{auth}} \cup \mathcal{M}$ **do:**
  Initialize Attack Graph $AG \leftarrow$ CreateHighLevelGraph(seed); Identify $AccessType \leftarrow$ CheckRemoteAccess($\mathcal{E}$).
  $SessionID \leftarrow$ ReassignSessionID($AccessType$); Link child user session to parent service session.
  // Intra-Machine Backward & Forward Traversal with Pseudo-Edges:
  $(E_\alpha, G_\alpha) \leftarrow$ TraverseBackward($SessionID, \mathcal{E}, \mathcal{L}_{\text{pers}}, \mathcal{L}_{\text{hijack}}, \mathcal{L}_{\text{portfwd}}$).
  $(E_\kappa, G_\kappa) \leftarrow$ TraverseForward($SessionID, \mathcal{E}, \mathcal{L}_{\text{pers}}, \mathcal{L}_{\text{hijack}}, \mathcal{L}_{\text{portfwd}}$).
$AG \leftarrow AG \cup G_\alpha \cup G_\kappa$.
// Cross-Machine & Cross-Host Network Tracing:
if cross-host socket / SSH connection is involved **then** Trace across enterprise network hosts via network provenance edges.
// Multi-Stage Lateral Movement Resolution:
if remote logon session detected **then** Correlate remote host provenance DAGs; Map enterprise MITRE ATT&CK TTPs.
// ATT&CK Annotation and Threat Score Calculation:
Annotate edges with MITRE TTPs; Compute $TS_{\text{edge}} = \sum w_i[Freq(teq_i) + Var(tac_i)]$.
Compute $TS_{\text{total}} = \sum TS_{\text{edge}} \cdot (1 + DA) \cdot Criticality$.
$\mathcal{L}_{\langle AG,TS\rangle} \leftarrow \mathcal{L}_{\langle AG,TS\rangle} \cup (AG, TS_{\text{total}})$.

return $\mathcal{L}_{\langle AG,TS\rangle}$ sorted descending by TS_{total}.

17. Implementation: Core System Modules & Architecture

1. Core Engineering Modules & Functional Pipeline

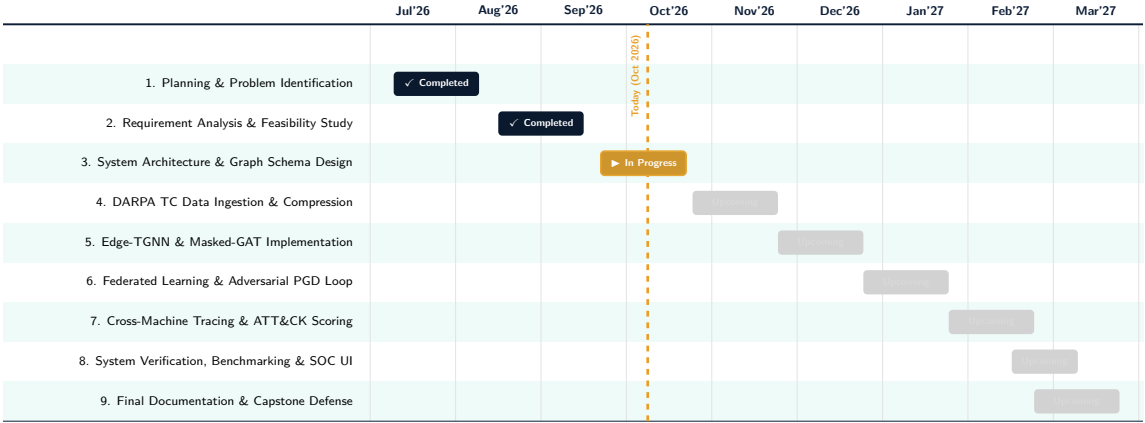
- **Kernel Ingestion & Parsing Engine:**
  Low-overhead audit probe hooks (Linux `auditd`, CamFlow LSM hooks, Windows Sysmon v14). Ingests raw audit logs and converts to CDM JSON/Avro records in real time.
- **Semantics-Preserving Event Compactor:**
  High-throughput Rust/Python microservice evaluating execution state equivalence and collapsing repetitive I/O loops (reduces raw volume $> 85\%$).
- **Dual-Tier AI Detection Server:**
  PyTorch Geometric microservice running lightweight Edge-TGNN (79.63 KB) on edge gateways and centralized self-supervised Masked-GAT on GPU cluster.
- **Whole-Network Causal Tracing Engine:**
  Logon-session partitioned DFS/BFS crawler inserting pseudo-edges to bridge persistence, token hijacking, and reverse proxy tunnels across hosts.

17. Implementation: Technology Stack & Frameworks

2. Technology Stack, Libraries & Frameworks

- **Deep Learning & Graph Frameworks:**
  Python 3.10+, PyTorch v2.2.0, PyTorch Geometric (PyG v2.5.0), DGL v1.1, SciPy v1.12 (K-d Tree spatial index), NumPy, and Scikit-Learn.
- **Privacy-Preserving Federated Cryptography:**
  TenSEAL CKKS Homomorphic Encryption for encrypted parameter aggregation in Federated Learning without gradient or data leakage.
- **Dual-Tier Database & Graph Storage Engines:**
  Neo4j Enterprise 5.x with Cypher query interface and APOC graph algorithms for high-speed DAG indexing; **PostgreSQL 16** for metadata and relational state.
- **Telemetry Sensors & Serialization:**
  FastAvro v1.9 for binary CDM decoding, Linux Auditd, CamFlow v0.5 LSM hooks, and Windows Sysmon v14 with Event Forwarding.
- **Forensic Dashboard & API:**
  FastAPI async REST API backend; **Cytoscape.js** interactive graph canvas for SOC triage and multi-stage DAG exploration.

18. Project Work Plan & Gantt Chart Roadmap



Completed **In Progress (Now)** **Upcoming** **Current Milestone (Oct 2026)**

19. References

[1] **Osei Isaac, Benjamin Appiah, Daniel Commey, Kwabena Owusu-Agyemang, Michael Asante, and Benjamin Hayfrom Acquah**, "EdgeFence: Federated Temporal Graph Neural Networks for Lightweight, Adversarial Malware Detection in Distributed Edge Networks," *International Journal of Information Security*, vol. 25, p. 88, 2026. DOI: 10.1007/s10207-026-01237-x

[2] **Mengjiao Cui, Zhengwei Jiang, Shuai Li, Chunyan Ma, Kai Zhang, Peian Yang, and Huamin Feng**, "MGDA: A provenance graph-based framework for threat detection and attack scenario reconstruction," *Computer Networks*, vol. 274, p. 111806, 2026. DOI: 10.1016/j.comnet.2025.111806

[3] **Qi Liu, Kaibin Bao, and Veit Hagenmeyer**, "Commander: A robust cross-machine multi-phase Advanced Persistent Threat detector via provenance analytics," *Journal of Information Security and Applications*, vol. 91, p. 104057, 2025. DOI: 10.1016/j.jisa.2025.104057

[4] **Roberto Canonico, Francesco Lista, Annalisa Navarro, Giancarlo Sperlí, and Andrea Vignali**, "Threat detection in reconfigurable Cyber–Physical Systems through Spatio-Temporal Anomaly Detection using graph attention network," *Computers & Security*, vol. 156, p. 104509, 2025. DOI: 10.1016/j.cose.2025.104509

[5] **Parth Atulbhai Gandhi, Prasanna N. Wudali, Yonatan Amaru, Akansha Shukla, Yuval Elovici, and Asaf Shabtai**, "SHIELD: Advanced persistent threats detection and intelligent explanation using large language models," *Engineering Applications of Artificial Intelligence*, vol. 181, p. 115443, 2026.

[6] **Francesco Ferazza, Cosimo Melella, Konstantinos Mersinas, Ricardo Lugo, and Rain Ottis**, "From logs to tactics: unsupervised reconstruction of APT campaigns with MITRE-enriched meta-alerts," *International Journal of Information Security*, vol. 25, p. 79, 2026.

[7] **Gisung Kim, Seungmin Lee, and Sehun Kim**, "A novel hybrid intrusion detection method integrating anomaly detection with misuse detection," *Expert Systems with Applications*, vol. 41, no. 4, pp. 1690–1700, 2014.

[8] **Haiyang Diao, Xiang Li, Wan Zhang, Yan Sibyl Yu, Ye Wang, and Jing Zhang**, "Self-supervised graph neural networks for network intrusion detection in cloud-edge," *Journal of Cloud Computing*, vol. 15, no. 1, p. 24, 2026.

19. References (Contd.)

[9] **Eslam Amer, Shaker El-Sappagh, Tamer Abuhamad, Bander Ali Saleh Al-Rimy, and Alaa Mohasseb**, "GraphShield: Advanced dynamic graph-based malware detection using graph neural networks," *Expert Systems with Applications*, vol. 268, p. 129812, 2025.

[10] **Fargana Abdullayeva and Suleyman Suleymanzade**, "Cyber security attack recognition on cloud computing networks based on GCN and GraphSAGE models," *Results in Control and Optimization*, vol. 16, p. 100423, 2024.

[11] **Rana Abu Bakar, Lorenzo De Marinis, Filippo Cugini, and Francesco Paolucci**, "FTG-Net-E: A hierarchical ensemble graph neural network for DDoS attack detection," *Computer Networks*, vol. 248, p. 110508, 2024.

[12] **Md Shakil Siddique, Md. Ashikur Rahman Khan, Ishtiaq Ahammad, Nishu Nath, Joysri Rani Das, and Fardowsi Rahman**, "Intelligent intrusion detection for cyber-physical systems using GAN-LSTM," *Franklin Open*, vol. 11, p. 100281, 2025.

[13] **Ali Yilmaz, Resul Das, Mehmet Ozdem, and Berk Canberk**, "Knowledge graph visualization and anomaly detection using graph neural networks," *Computer Networks*, vol. 278, p. 112632, 2026.

[14] **Giancarlo Iannizzotto, Andrea Nucita, and Andrea Siragusa**, "Robust, efficient detection of network intrusion attacks: A large language model approach," *Computer Networks*, vol. 276, p. 112559, 2026.

[15] **Abdullah Al Mamun, Harith Al-Sahaf, Ian Welch, and Masood Mansoori**, "Detection of advanced persistent threat: A genetic programming approach," *Applied Soft Computing*, vol. 167, p. 112447, 2024.

[16] **Mohammed Alshehri**, "Dynamic cyber deception using AI-driven adaptive honeypot networks," *Ain Shams Engineering Journal*, vol. 17, no. 3, p. 104061, 2026.



Thank You

Questions & Discussion

Muhammed Nasih K P (KNP23CS074) **Sumayya Ershad** (KNP23CS107)

Najmunniza N (LKNP20CS056) **K.S. Mahadev** (KNP23CS066)

Department of Computer Science & Engineering
College of Engineering, Karunagappally

Guide: Mrs. SYAMA S R, Assistant Professor